Boycode Bootcamp 2026

Team 7 Practical Assignment Report

Exploiting Application-Based Vulnerabilities

Instructor: Michael Opara | Due: October 16, 2026

Student Names: Kazeem Abubakri, Oladipo Hezekiah, Oluwatimilehin Abisogun

Date: October 16, 2026

Targets: OWASP Juice Shop (local Docker instance, http://localhost:3000) and DVWA (local Docker instance, http://localhost:8081)

Authorization: All testing was done against our own self-hosted, isolated lab instances inside a Kali Linux VM (VirtualBox). Nothing outside that VM was touched.

1. Scope and Methodology

For this assignment we set up two deliberately vulnerable apps locally — OWASP Juice Shop and DVWA — and ran them as Docker containers inside my Kali VM so everything stayed isolated from anything real. We tried a public Juice Shop demo link first (juice-shop.herokuapp.com) but it was dead, so we then switched to running my own copy locally, which also matched the assignment's requirement to only test authorized, isolated targets.

1.1 Lab Setup

- Juice Shop: sudo docker run -d -p 3000:3000 bkimminich/juice-shop
- DVWA: sudo docker run -d -p 8081:80 vulnerables/web-dvwa, security level set to Low so the vulnerabilities would actually be exploitable for this exercise
- Burp Suite Community Edition as the intercepting proxy (127.0.0.1:8080)
- FoxyProxy in Firefox to route traffic to Burp — we also had to flip network.proxy.allow_hijacking_localhost to true in about:config, since Firefox doesn't proxy localhost traffic by default and we couldn't see any of my own requests in Burp until we corrected that.
- sqlmap for the automated SQL injection comparison in Task 6.1

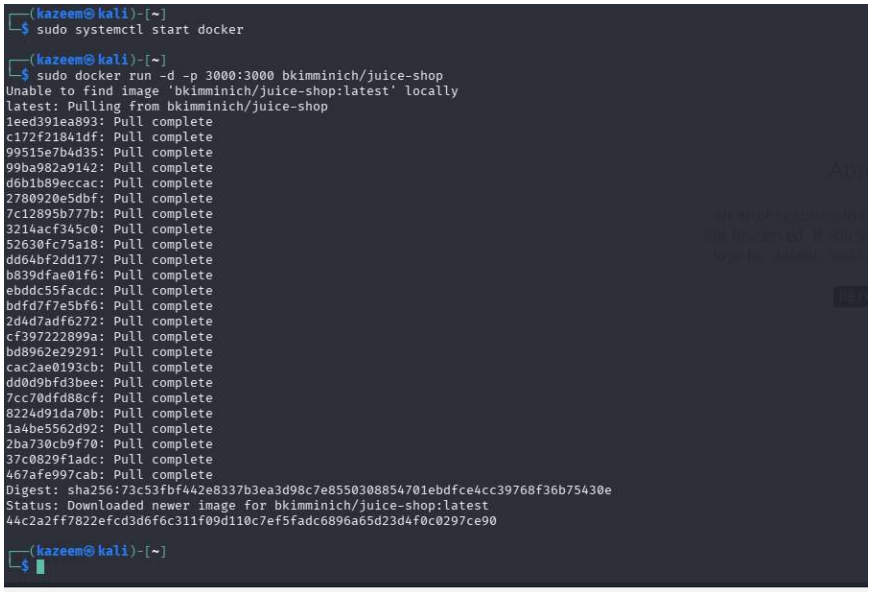


Figure 1.1 — Juice Shop Docker image pulled and container started.



Figure 1.2 — DVWA Docker image pulled and container started on port 8081.

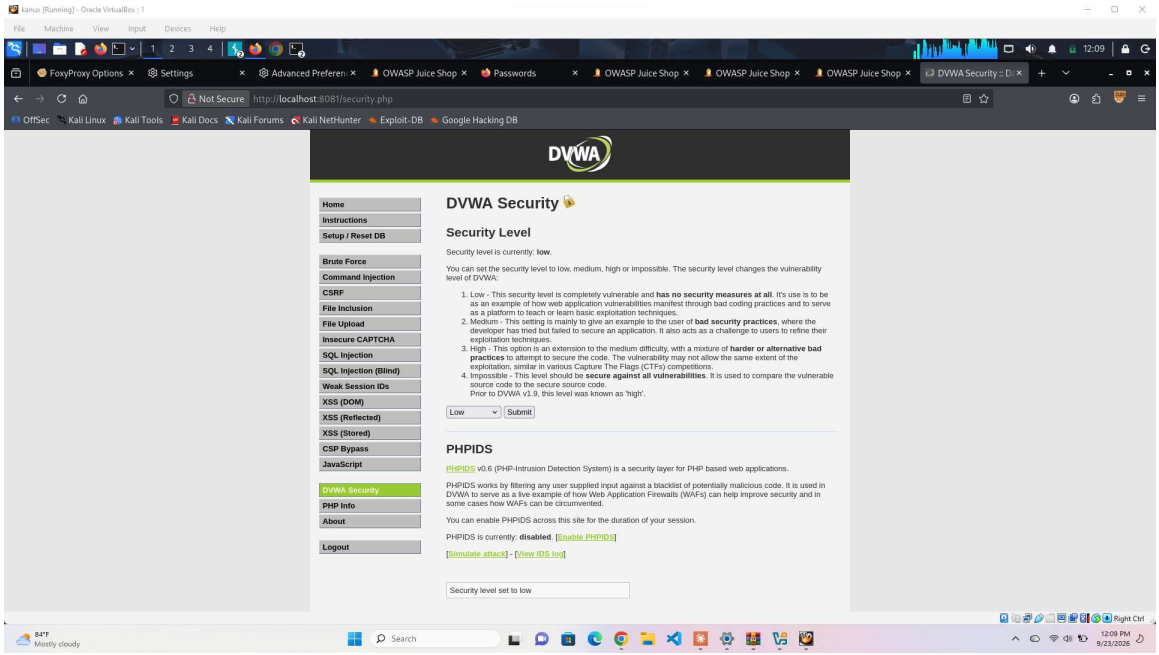


Figure 1.3 — DVWA security level set to Low before testing.

1.2 Boundaries

- Only ourown local containers were tested — no public sites
- Only fake/lab data used throughout
- For the command injection and file upload parts, we kept everything harmless — just proving the code runs (e.g. printing whoami), not doing anything destructive or leaving a working web shell behind.

2. Task 6.1 — SQL Injection: Manual and Automated

2.1 What I Found

Vulnerability: SQL Injection — Authentication Bypass

OWASP Category: A03:2021 – Injection

Where: POST /rest/user/login (Juice Shop)

The login form doesn't sanitize the email field at all, so it's possible to log in as basically anyone — including the admin — without knowing a real password.

2.2 Manual Exploitation

We put ' OR 1=1-- in the email box and just typed a random password (1234). We weren't totally sure it would work on the first try, but it did.

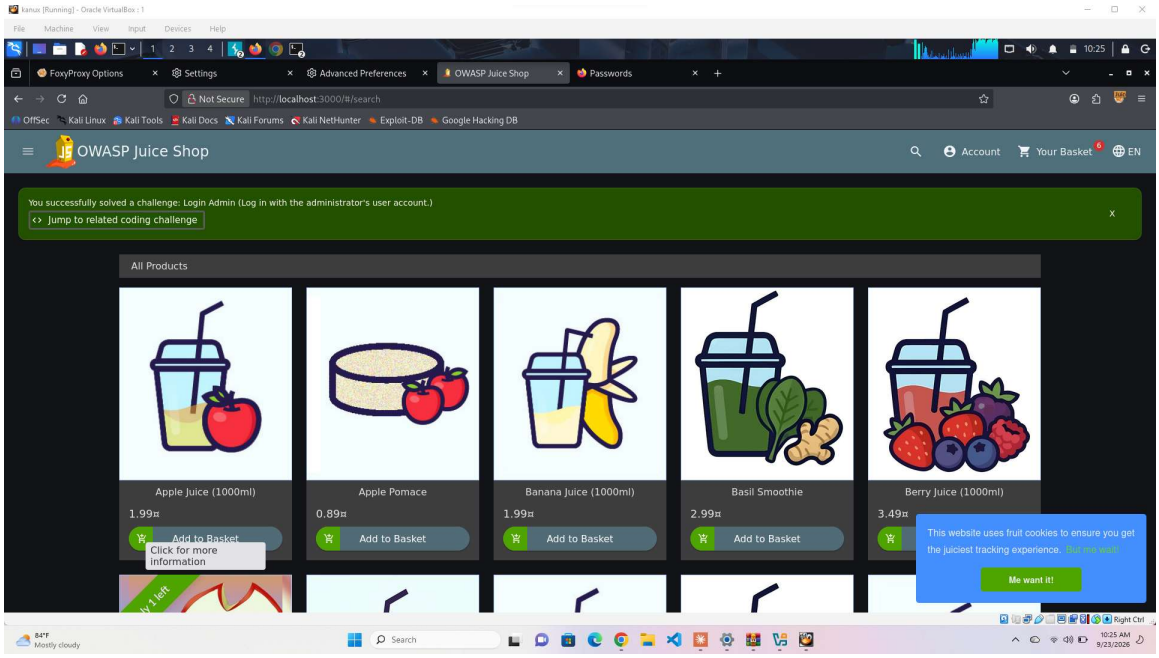


Figure 2.1 — Juice Shop's own challenge tracker confirms "Login Admin" was solved.

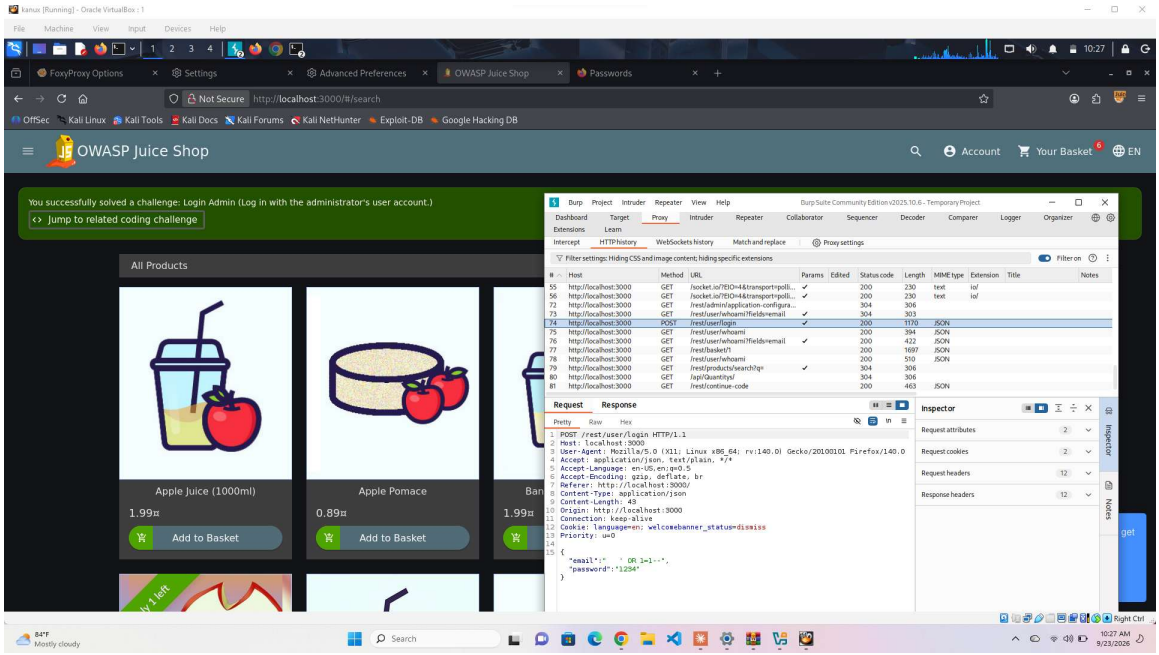


Figure 2.2 — Burp capture of the request showing the payload in the JSON body.

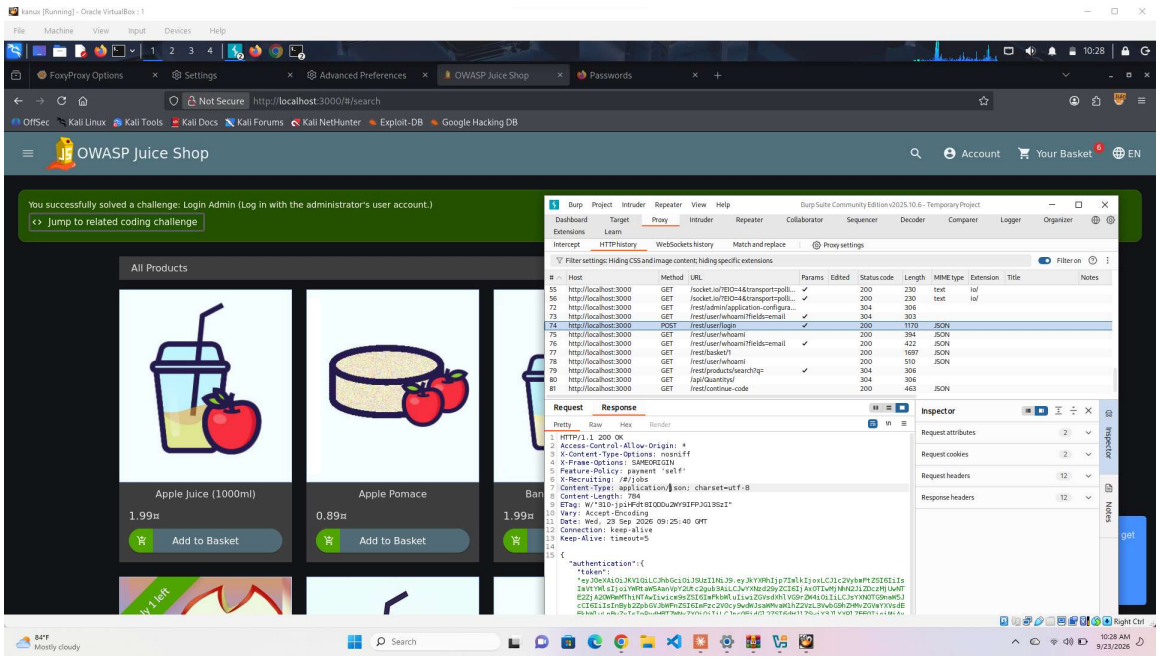


Figure 2.3 — The response: 200 OK with a full JWT token, so the server logged me in.

2.3 Root Cause

This only works if the backend is building the SQL query by just gluing strings together instead of using parameters, something like:

```
SELECT * FROM Users WHERE email = '" + email + "' AND password = '" + password + "'
```

Our payload turns that into:

```
SELECT * FROM Users WHERE email = '' OR 1=1--' AND password = '1234'
```

The apostrophe closes the string early, OR 1=1 is always true so it matches every row, and -- comments out the rest of the line (including the password check). So the database just hands back a valid user — whichever one it hits first — no matter what password I typed.

2.4 Impact

This is about as bad as it gets — anyone can log in as admin with zero effort. Full account takeover, admin panel access, everything. We'd call this Critical.

2.5 Trying sqlmap

We saved the login request from Burp and pointed sqlmap at it to compare manual vs automated testing, keeping the settings low-impact like the assignment asked:

```
sqlmap -r login_request.txt --level=3 --risk=1 -p email
```

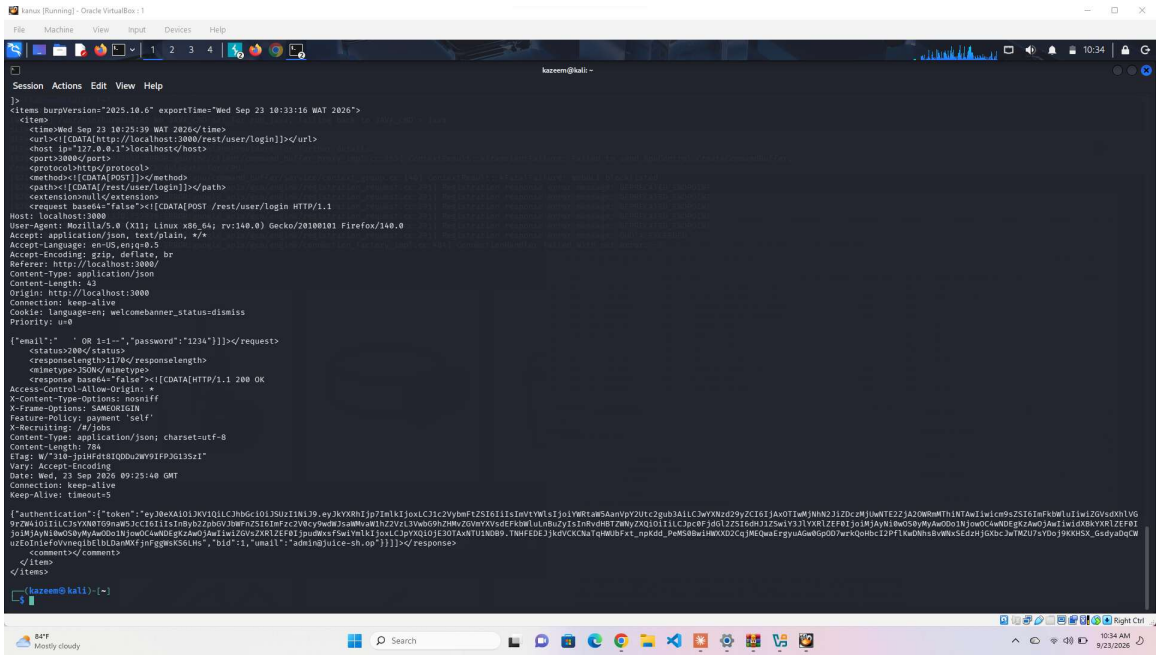


Figure 2.4 — sqlmap's first pass: it decides the email parameter "might not be injectable" after trying a bunch of boolean-based techniques.

That surprised me since we'd already proven it manually, so we tried again with much more aggressive settings and told it to ignore the 401 responses it kept getting:

```
sqlmap -r login_request.txt -p email --technique=B --level=5 --risk=2 --ignore-code=401
```

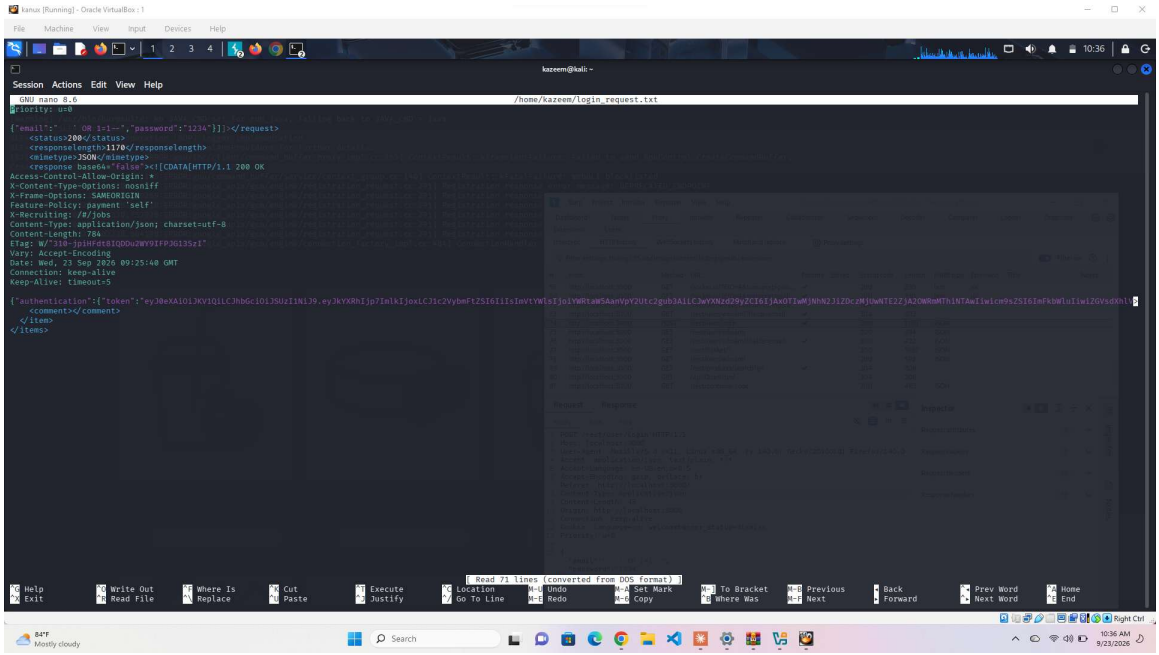


Figure 2.5 — Still no luck — sqlmap reports the parameter isn't injectable even at max level/risk.

Manual vs Automated — what we took from this

Honestly this was the most interesting part of the whole task. We expected sqlmap to confirm what we'd already found in seconds, but it just couldn't see it. My best guess after reading up on it is that sqlmap's boolean-based detection works by comparing small differences between a "true" response and a "false" response (usually length), but Juice Shop's login either gives you a full 200 with a token or a flat 401 — there's no in-between for sqlmap to notice a pattern in.

So the lesson for me here is that automated tools aren't magic — they're looking for specific patterns, and if a vulnerability doesn't behave the way the tool expects, it can walk right past it and tell you "all good" when it really isn't. If we'd only run sqlmap and trusted the result, we'd have completely missed a critical vulnerability we found by hand in one try. It also goes the other way — the assignment's question about why automated tools carry extra risk makes sense to me now too, since sqlmap was firing off dozens of requests per second across every technique it tried, which on a real system could definitely trip a WAF, get an account locked out, or just hammer the server if you don't cap the risk/level like we did here.

2.6 How we'd Fix It

- Use parameterized queries / prepared statements instead of concatenating strings into SQL
- Give the login service a database account with the least privilege it actually needs
- Add input validation too, just as a backup layer
- Make login failures return a generic error instead of anything that hints at what the query looked like

3. Task 6.2 — Cross-Site Scripting (Reflected, Stored, DOM)

3.1 What I Found

Confirmed: DOM-based XSS through the search bar

OWASP Category: A03:2021 – Injection

Stored XSS: Tried it on two different fields (reviews and delivery address) and couldn't get it to fire on either — both looked properly protected

3.2 DOM XSS — What I Did

we typed <iframe src="javascript:alert('xss')"> straight into the search bar.

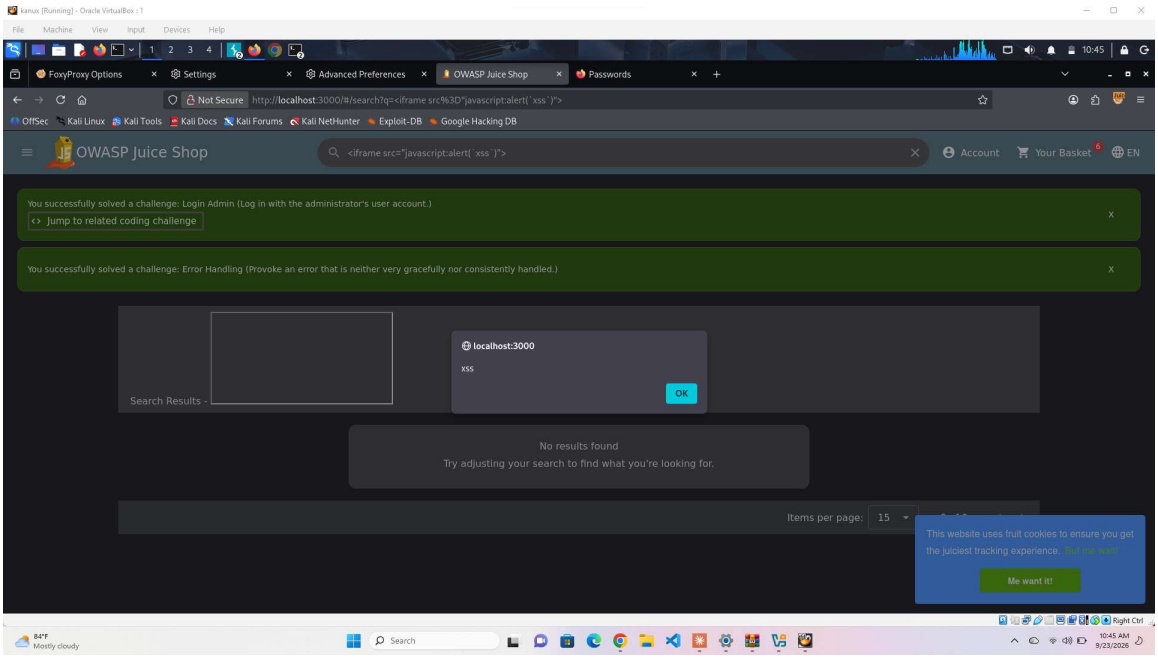


Figure 3.1 — Alert pops up right away, and Juice Shop credits both the DOM XSS and Error Handling challenges.

We wanted to check whether this was actually reflected off the server or purely client-side, so we checked what Burp captured for the request.

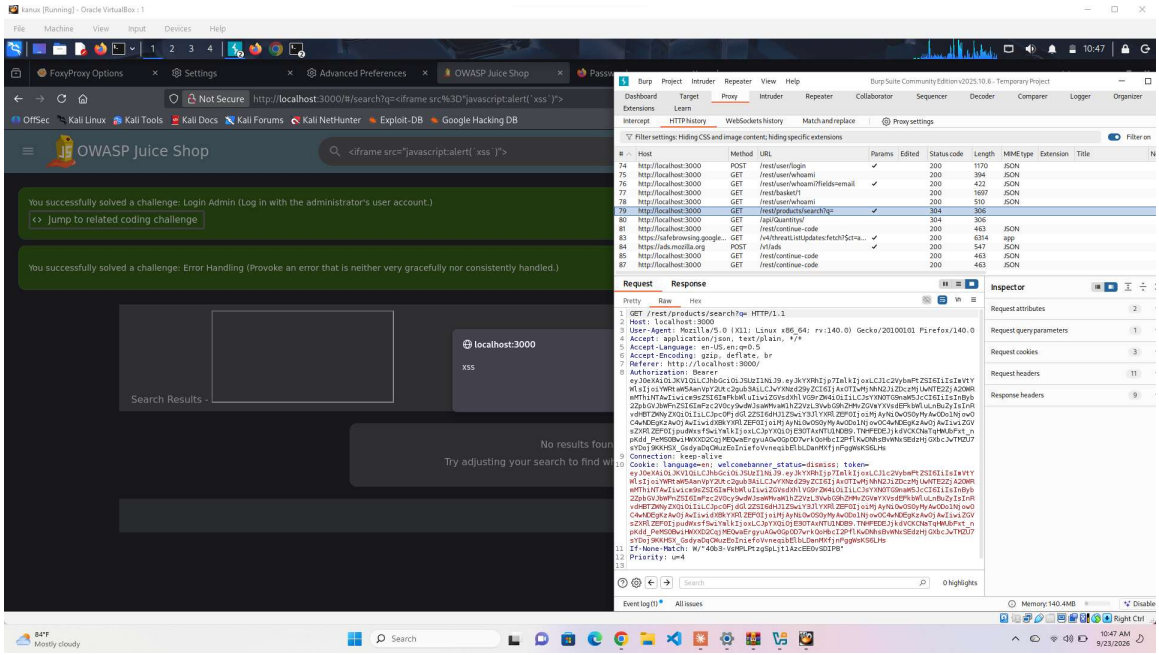


Figure 3.2 — The GET /rest/products/search request in Burp.

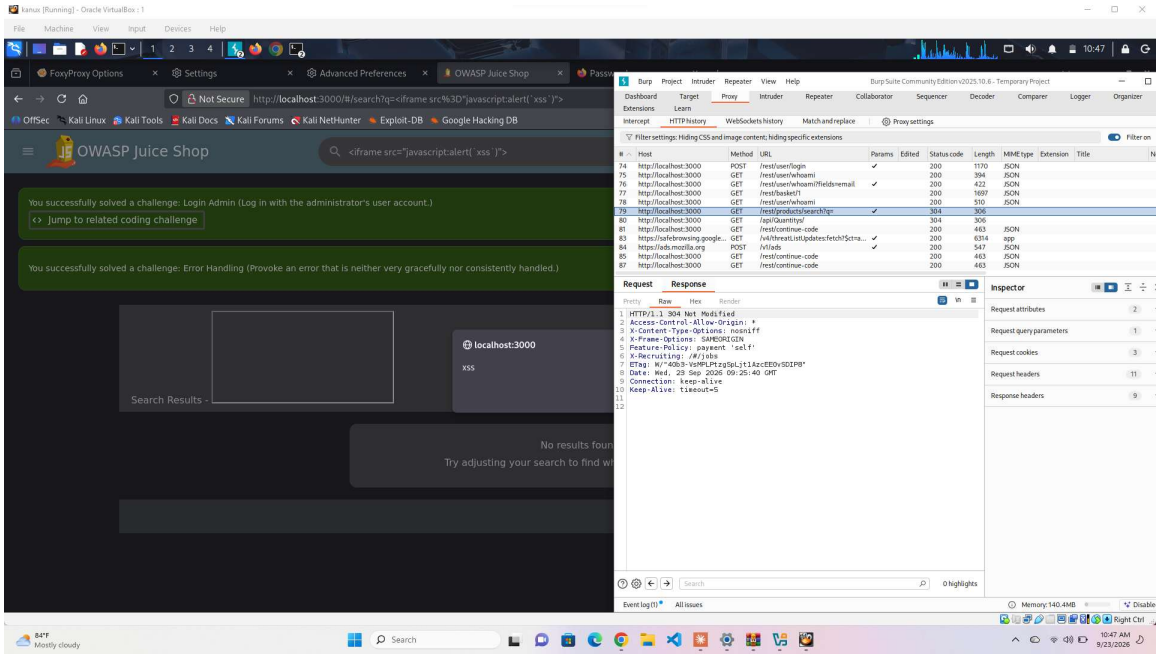


Figure 3.3 — And the response: 304 Not Modified, completely empty body. Our payload never shows up anywhere in what the server sent back.

That told me this isn't reflected XSS at all — it's DOM-based. Juice Shop is a single-page app that uses a # in the URL, and browsers never send anything after a # to the server. So Angular is reading the search term straight out of the URL on the client side and dropping it into the page without cleaning it up first — the server never even sees it.

3.3 Trying Stored XSS

We tried the payload <img src=x onerror=alert('stored-xss')> in two places: a product review and my delivery address. Both times it saved fine, but when we reloaded, it just showed up as plain text instead of running.

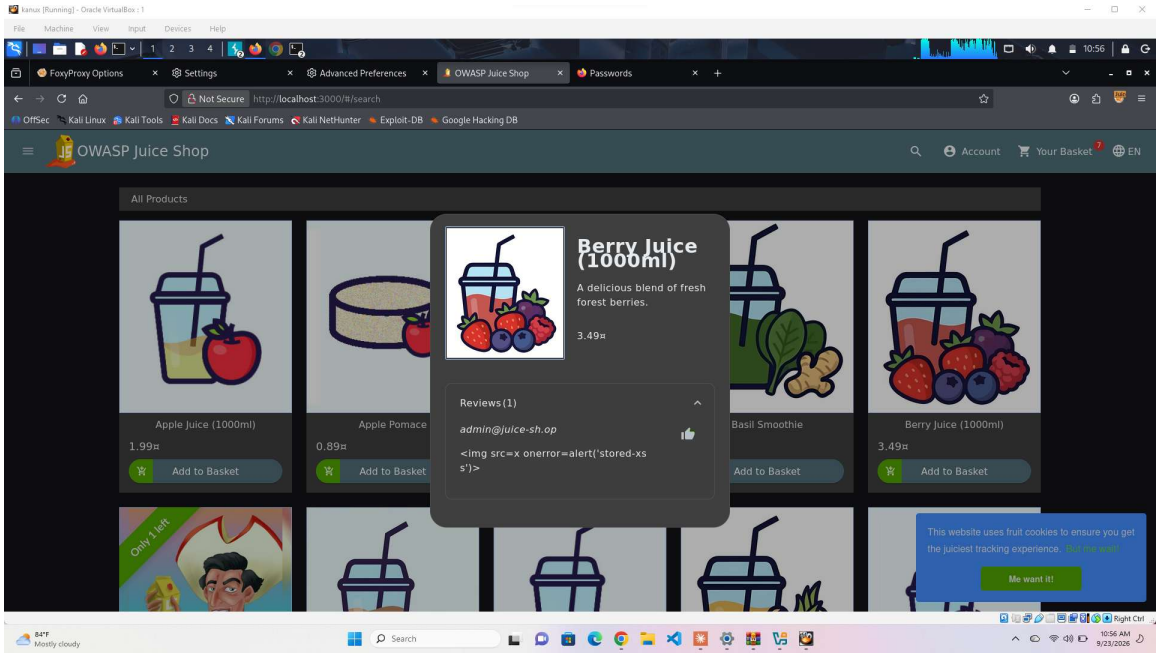


Figure 3.4 — Our payload sitting in the Berry Juice review, escaped and harmless.

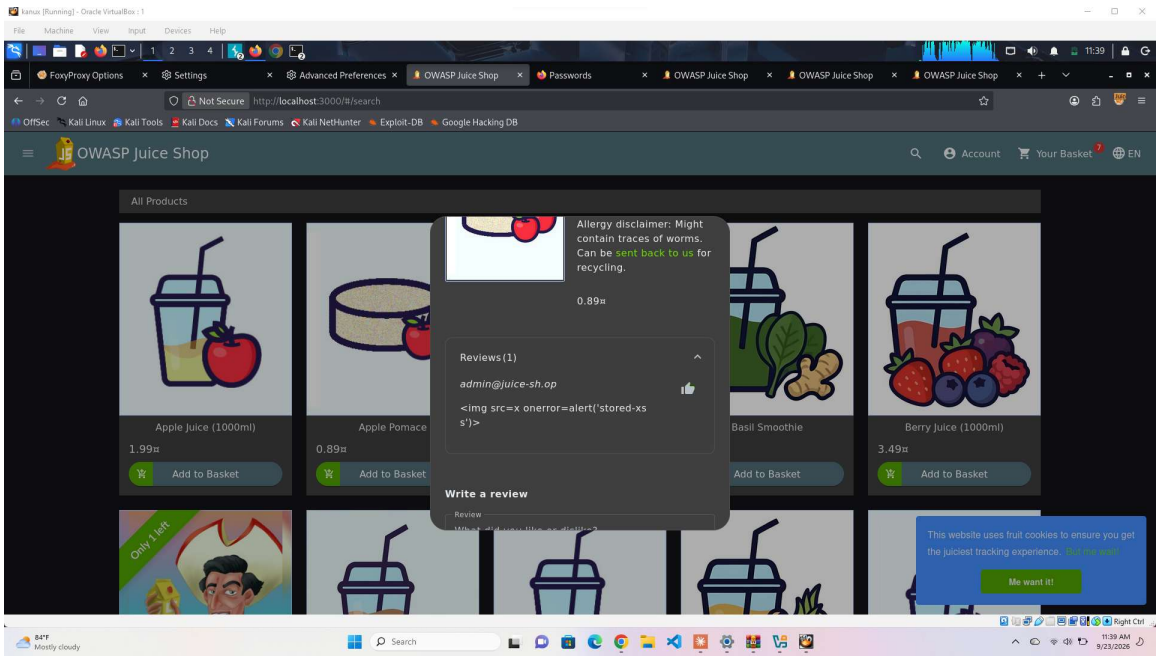


Figure 3.5 — Tried again on a totally different product (Apple Pomace) to rule out a fluke — same result.

we also tried the Customer Feedback form, since that's actually the field Juice Shop calls out as its intended “stored XSS” challenge, but the Submit button just wouldn't activate no matter how many times we filled every field correctly (comment, rating, CAPTCHA). Our mentor apparently ran into the exact same dead end, so we’re fairly confident that's just a broken part of this build rather than something we did wrong.

3.4 Impact

The DOM XSS is a real problem — someone could send a link with the payload baked into the URL, and the moment a victim clicks it, arbitrary JavaScript runs in their browser, no server interaction needed at all. That could be used to steal session data or trick a user into doing something they didn't mean to. On the flip side, we think it's worth noting the review and address fields held up fine — they properly encode whatever you type before showing it back, so not everything here is broken.

3.5 Fixes

- Sanitize the search 'q' value before it gets written into the page, or use Angular's built-in DomSanitizer instead of an unsafe binding
- Add a Content-Security-Policy header to block inline scripts as a backup layer
- Set HttpOnly and SameSite on cookies so even if XSS happens, session theft is harder
- Whatever's being done right on the reviews/address fields should be applied consistently everywhere else too

What we took from this part

This drove home for me that one bad spot is enough to undo everything else being done right — the search bar being unsafe doesn't get any less dangerous just because the review box next to it is fine. It's less about finding “the” vulnerability and more about checking every input consistently, since attackers only need one gap.

4. Task 6.3 — Broken Access Control and IDOR

4.1 Roles and Objects

- Object: shopping basket, identified by a plain sequential number (/rest/basket/:id)
- Accounts used: admin@juice-sh.op (the account I got into via the SQLi in Task 6.1) and a fresh account I registered myself, testuser@test.com

4.2 IDOR on the Basket

While logged in as admin, my basket was ID 1. I grabbed that request in Burp, sent it to Repeater, and just changed the 1 to a 2.

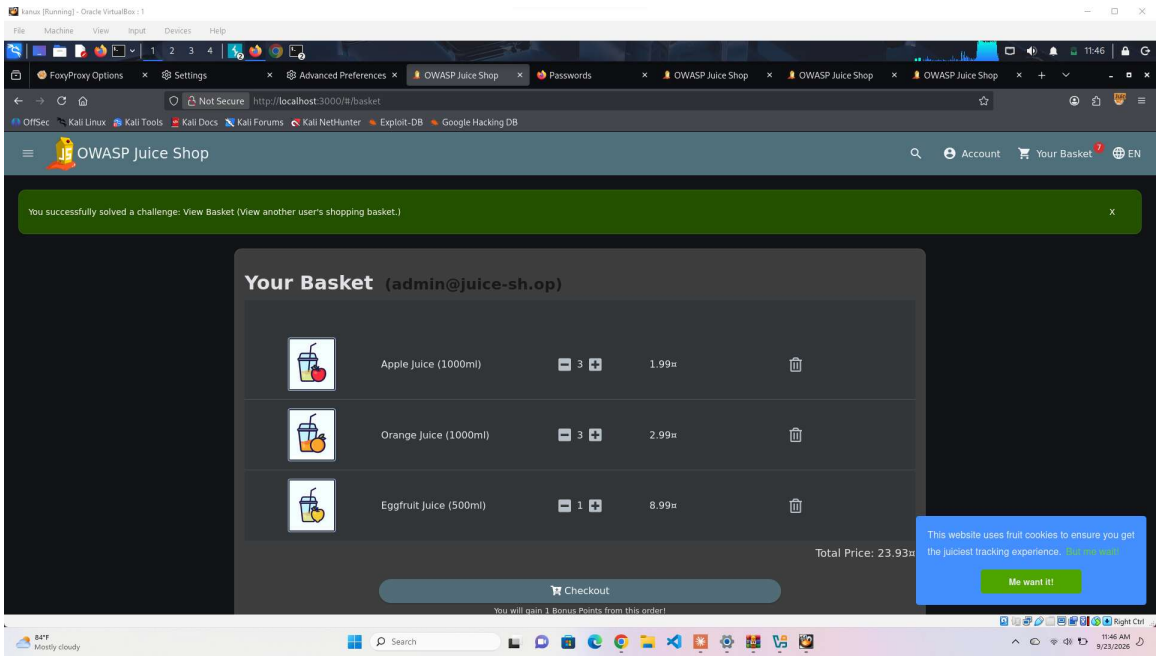


Figure 4.1 — Juice Shop confirms the "View Basket" challenge — I viewed someone else's basket.

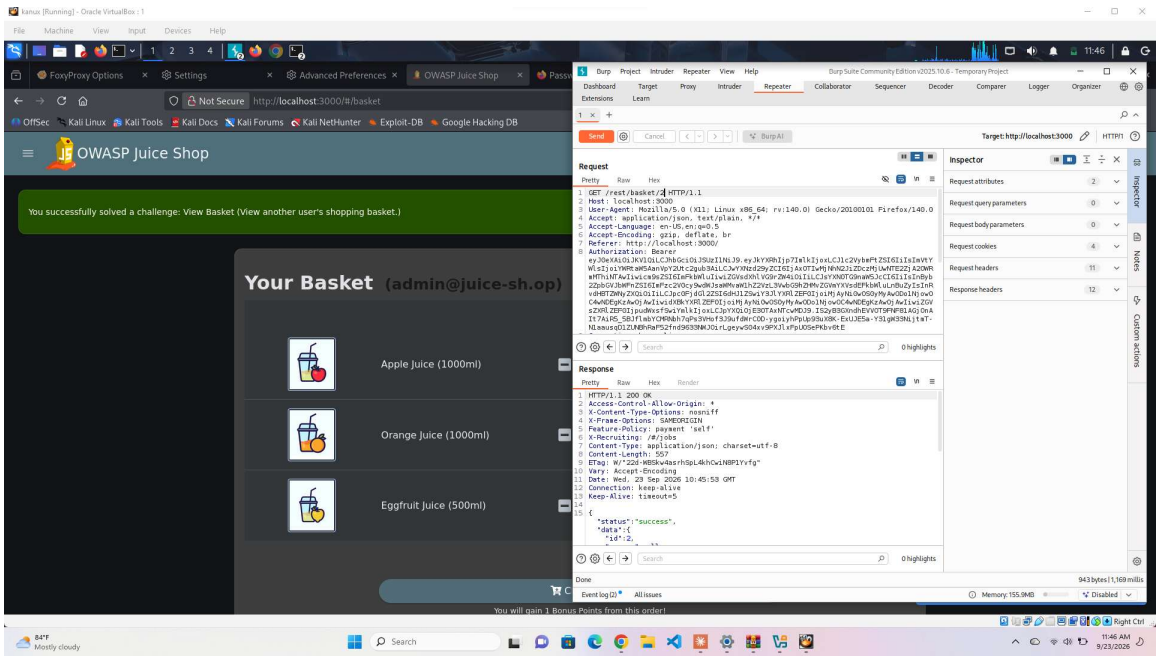


Figure 4.2 — GET /rest/basket/2 in Repeater, still using my own admin token, comes back 200 OK.

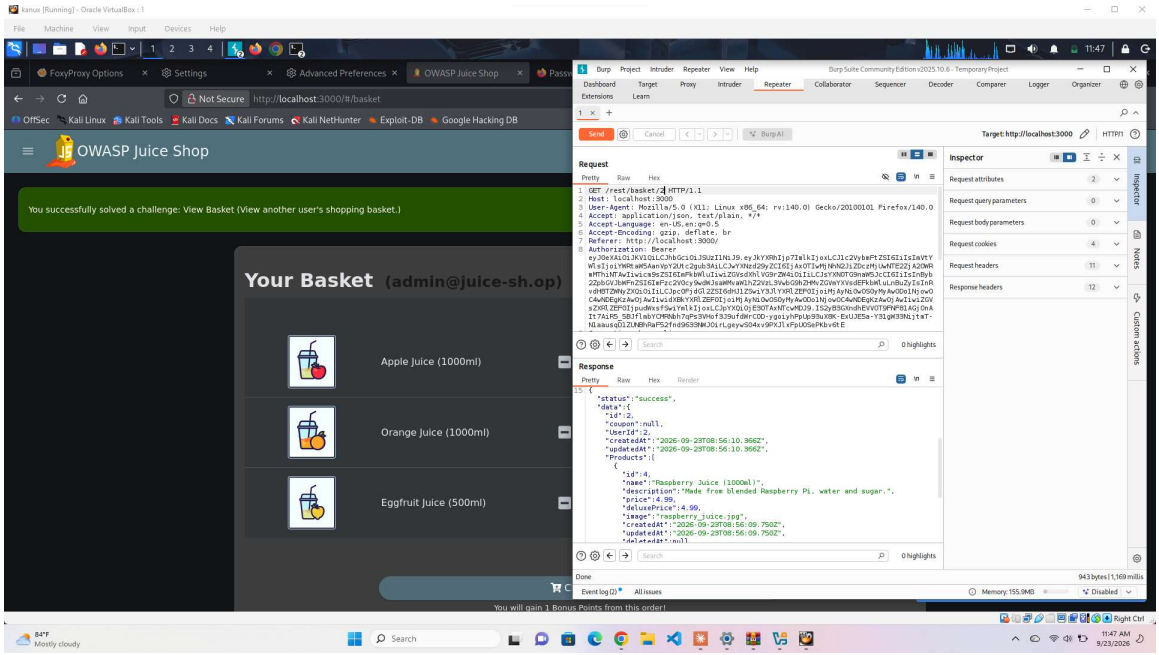


Figure 4.3 — And the response body shows UserId 2 and real products (Raspberry Juice) — definitely not my basket.

It should have returned a 403 since basket 2 has nothing to do with my account, but instead it just handed the data straight over. Since these IDs are just sequential numbers, anyone could loop through them and pull every user's basket without much effort at all.

4.3 Vertical Escalation — Admin Endpoint

I registered a normal account (testuser@test.com) and tried loading the admin page directly.

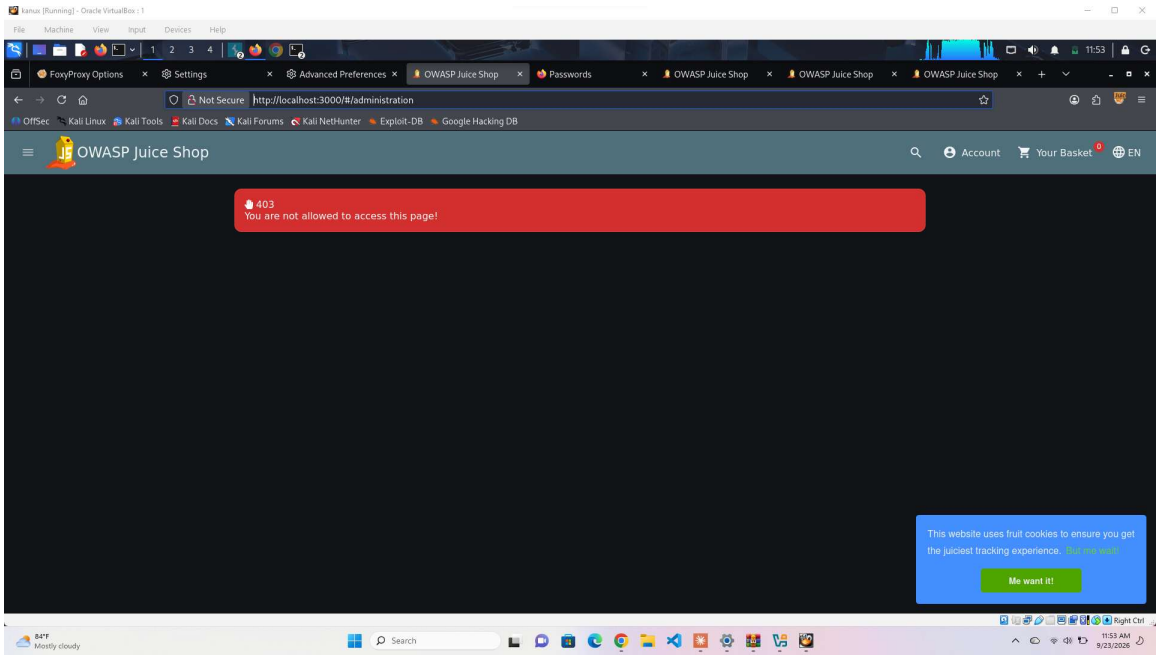


Figure 4.4 — Going to /#/administration as this regular user gives a proper 403 page — the frontend blocked me.

But I wanted to check if that 403 was a real security boundary or just a page redirect, so I checked what the actual API call behind that page was doing in Burp.

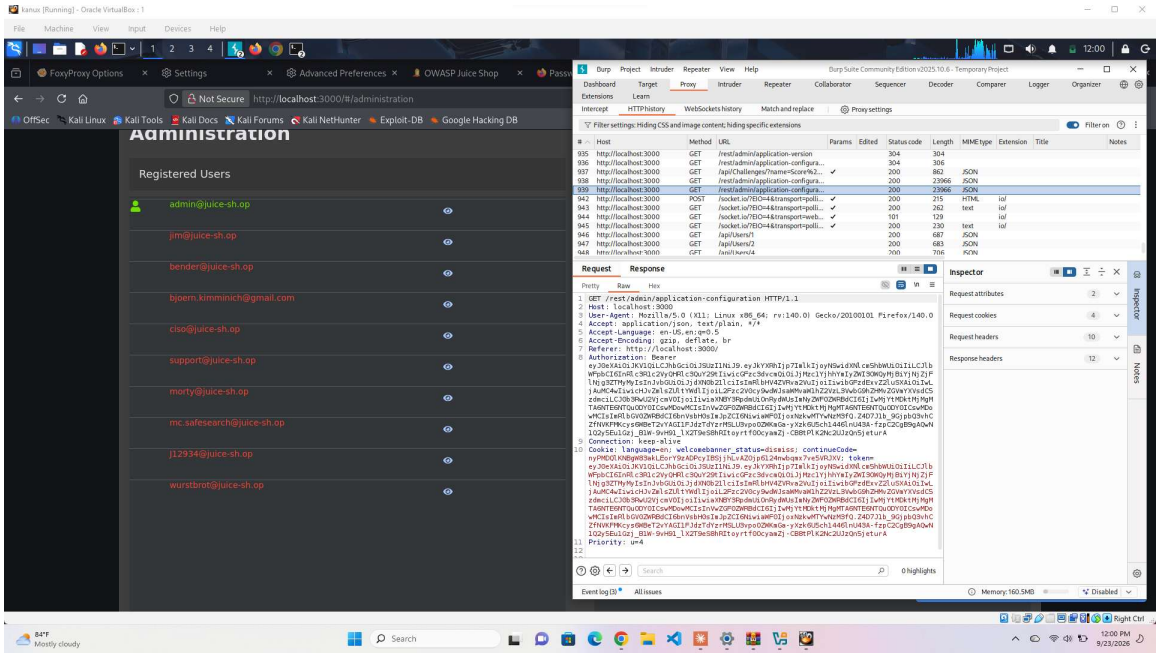


Figure 4.5 — GET /rest/admin/application-configuration, using my regular test account's token, still returns 200 OK with the full config data.

So the 403 page is really just cosmetic — it stops you clicking around in the UI, but the actual API underneath has no idea what role you are and will happily hand back admin data to anyone logged in at all. That felt like the sneakiest finding of the whole assignment, since it would be really easy to think you're protected just because the page shows an error.

4.4 Impact

Put together, these two findings mean any account — even one I just made up five minutes ago — can pull other users' private basket data and grab admin configuration data, all without ever needing to actually compromise a privileged account first.

4.5 Fixes

- Check ownership server-side on every request that returns an object, not just show/hide buttons on the frontend
- Apply that same check to updates and deletes, not only reads
- Every /rest/admin/* route needs an actual role check on the backend
- Non-sequential IDs (UUIDs) would help as an extra layer, though that alone doesn't fix the missing check
- A test that logs in as User A, requests User B's basket, and expects a 403 would catch this kind of regression going forward — same idea for admin routes and non-admin tokens

5. Task 6.4 — Command Injection / Insecure File Upload to RCE

5.1 What I Found

Environment: DVWA, security level Low

Confirmed: OS command injection through the ping tool, and unrestricted file upload leading to remote code execution

5.2 Command Injection

DVWA has a “ping a device” box. I tried tacking on an extra command after a real IP:

```
127.0.0.1 && whoami
```

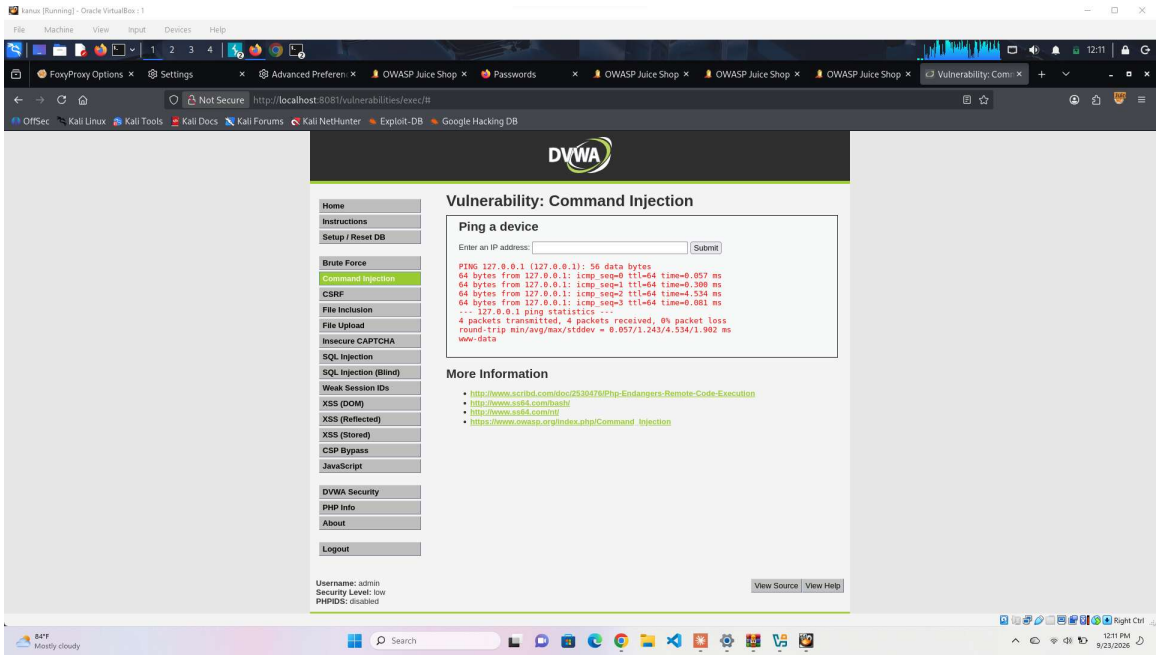


Figure 5.1 — The normal ping output shows up, then “www-data” right after it — my extra command actually ran on the server.

This only works because whatever the input box is doing is just shoving my text straight into a shell command, no checks at all for things like && or ; that let you chain extra commands on.

This is basically as bad as a vulnerability gets — full command execution as the www-data user. Depending on what that account can touch, this could go a lot further than just printing a username.

5.3 File Upload

I made a tiny, harmless PHP file that just prints a message — nothing malicious, no actual shell:

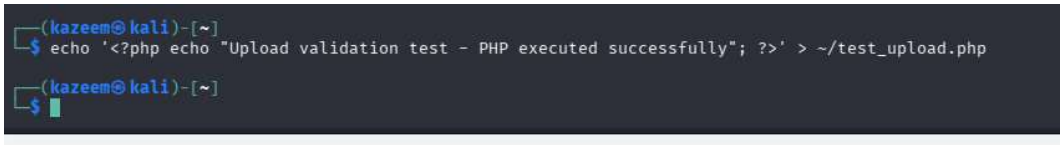


Figure 5.2 — Creating test_upload.php in the terminal.

Then uploaded it through DVWA's upload form.

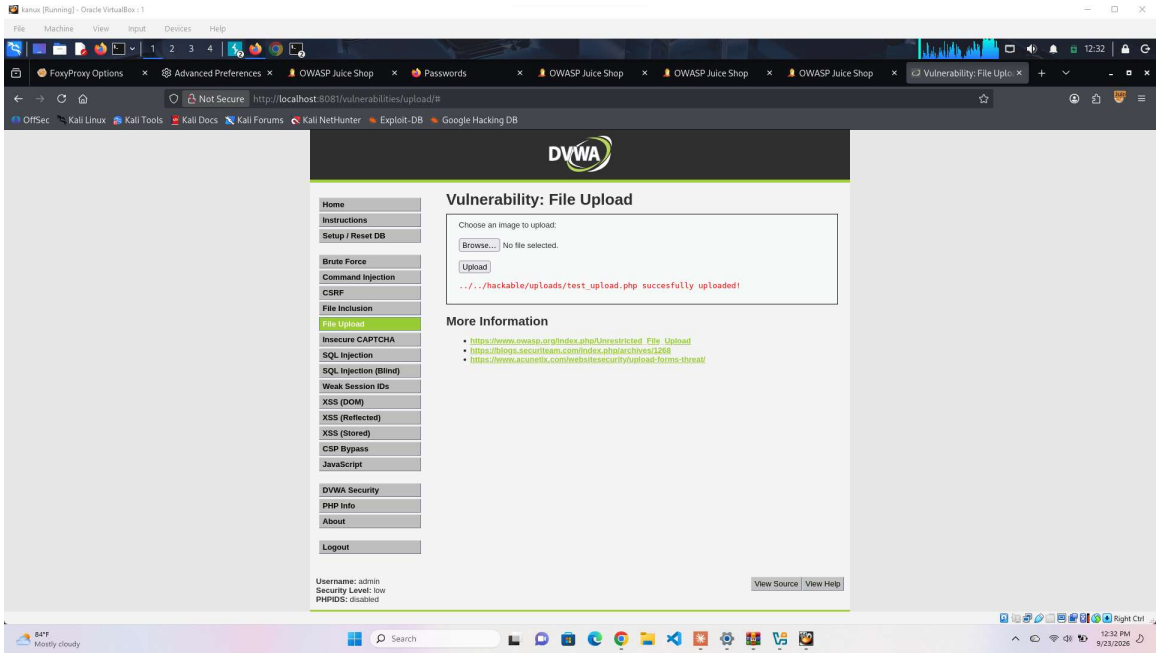


Figure 5.3 — It accepted the .php file with zero checks and saved it to /hackable/uploads/.

To confirm it wasn't just stored as a dead file, I opened it directly in the browser.

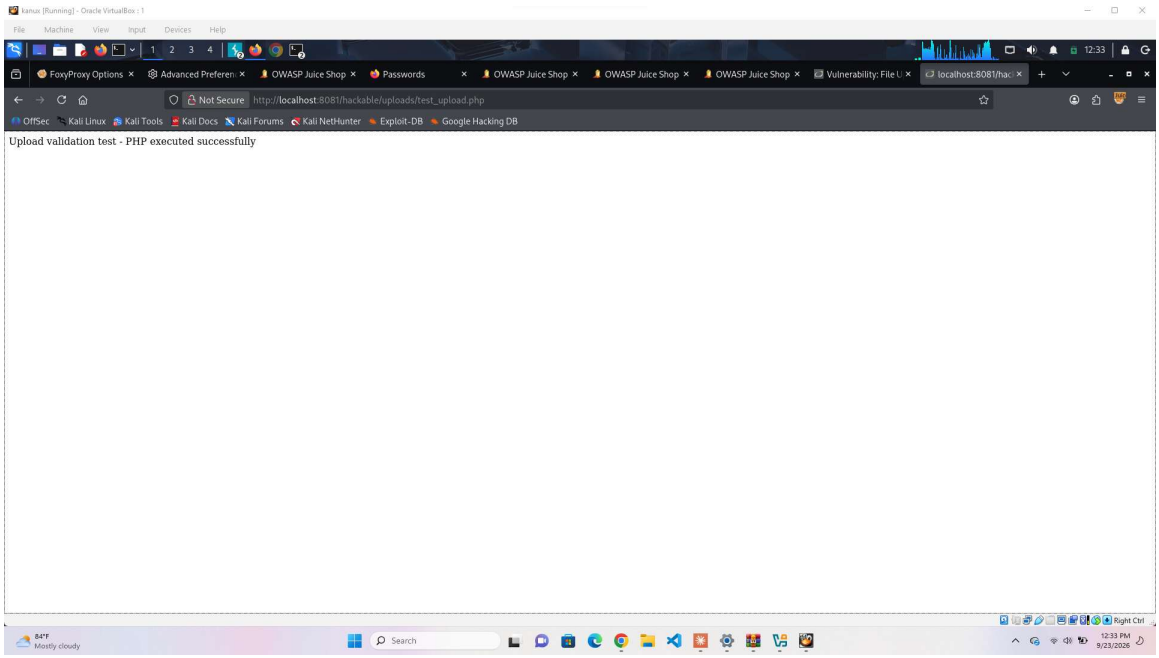


Figure 5.4 — "Upload validation test - PHP executed successfully" — the server actually ran it as code.

So the upload form doesn't check the file type, extension, or contents at all before saving it into a folder the server will happily execute PHP out of. In a real attack this same gap is how someone plants a working web shell — I kept mine to a single harmless print statement on purpose, per the assignment's rules about not leaving anything destructive or persistent behind.

5.4 Impact

Both of these are Critical — they're not just data leaks, they're straight-up remote code execution. Anyone who can reach these forms can run their own code on the server.

5.5 Fixes, Roughly in Order of Effort

Quick wins

- Only allow specific file extensions (jpg, png, gif) and reject everything else
- Actually check the file's real content/type server-side, not just trust whatever extension the client sends
- For the ping tool, stop building a shell string from user input — use a real networking call instead of shelling out where possible

Bigger fixes

- Keep uploaded files outside the web root, or turn off script execution in the uploads folder entirely
- Rename uploaded files to random names so they can't be guessed or targeted by extension
- If shelling out really is necessary, validate the input strictly (e.g. it has to look like a real IP) and call the command without going through a shell string at all
- Run the web server as a locked-down, low-privilege account so even a successful exploit doesn't get much

6. Reflection

This was a lot more hands-on than I expected going in, and a good chunk of the time actually went into just getting the lab environment working rather than the exploits themselves — getting FoxyProxy to actually route localhost traffic through Burp took a while to figure out, Burp crashed on me once mid-session, and I hit a dead public Juice Shop link before switching to running everything locally with Docker. None of that is glamorous, but it's honestly a real part of pentesting — half the job is just getting your tooling to cooperate before you even get to the fun part.

The SQL injection was the most satisfying because it worked first try by hand, and then genuinely surprised me by beating sqlmap at every setting I threw at it. That flipped something in my head about not blindly trusting scanner output. The XSS section taught me the opposite lesson — that a "no" from one field (reviews, address) doesn't mean the whole app is safe, since the search bar right next to it was wide open. The access control section was probably the most realistic-feeling one: seeing a proper-looking 403 page and then finding the actual API behind it didn't care at all was a good reminder that frontend restrictions aren't security. And the DVWA section made remote code execution feel very concrete instead of abstract — one unsanitized text box and one unchecked file upload were both enough to run arbitrary code on the server.

Overall, the big pattern I noticed across all four tasks is that these vulnerabilities weren't exotic or hard to find — they were all just missing basic checks (parameterize the query, encode the output, verify ownership, validate the input) that got skipped somewhere. That's a bit unsettling but also reassuring, since it means the fixes aren't complicated either, just consistent.

— End of Report —